

- RAG Learning

4.3 Fundamentals of Algorithms

Computer Science — AQA A-Level (7517) — Paper 1: On-screen Programming

Topic 4.3 builds directly on the data structures of 4.2: algorithms are the *procedures* that operate on those structures. Paper 1 examiners will ask you to **trace** an algorithm step-by-step, **compare** two algorithms for the same task, **state** the Big-O time and space complexity, and **justify** which algorithm best fits a given scenario. This note works through every named algorithm in the AQA specification — graph and tree traversals, Reverse Polish, searching, sorting, Dijkstra — with full worked traces and pseudocode suitable for exam answers.

4.3.1 Graph-Traversal Algorithms

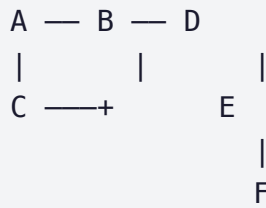
A graph traversal visits every reachable vertex starting from a given source. The two standard strategies differ only in which auxiliary data structure holds the *frontier* of vertices to process next.

Depth-First Search (DFS) explores as far as possible along one branch before backtracking. The frontier is held in a **stack** (explicit or via recursion). DFS discovers a deep path quickly but may miss shorter routes.

Breadth-First Search (BFS) explores all neighbours of a vertex before moving on to their neighbours. The frontier is held in a **queue**. BFS discovers vertices in non-decreasing order of *number of edges from the source*, giving shortest paths in unweighted graphs.

Example Graph

All the traces below use this undirected, unweighted graph with six vertices. Adjacency lists are given in alphabetical order, which determines tie-breaking in both algorithms.



Adjacency lists:

A: [B, C]
 B: [A, C, D]
 C: [A, B]
 D: [B, E]
 E: [D, F]
 F: [E]

DFS Pseudocode (Recursive)

```

PROCEDURE DFS(graph, v, visited)
  ADD v TO visited
  OUTPUT v
  FOR EACH neighbour IN graph[v]
    IF neighbour NOT IN visited THEN
      DFS(graph, neighbour, visited)
    ENDIF
  ENDFOR
ENDPROCEDURE

```

DFS Pseudocode (Iterative, Explicit Stack)

```

PROCEDURE DFS_iterative(graph, start)
  DECLARE stack : Stack
  DECLARE visited : Set
  stack.push(start)
  WHILE NOT stack.isEmpty() DO
    v ← stack.pop()
    IF v NOT IN visited THEN
      ADD v TO visited
      OUTPUT v
      FOR EACH neighbour IN REVERSE(graph[v])
        IF neighbour NOT IN visited THEN
          stack.push(neighbour)
        ENDIF
      ENDFOR
    ENDIF
  ENDWHILE
ENDPROCEDURE

```

```

        ENDIF
    ENDFOR
ENDIF
ENDWHILE
ENDPROCEDURE

```

Reversing the neighbour list before pushing is a common trick: because the stack is LIFO, pushing in reverse makes the iterative version visit vertices in the same order as the recursive version.

DFS Worked Trace — Starting from A

The *stack* column shows contents with the top on the right. *Visited* shows the output order.

Step	Action	Stack	Visited (output)
0	push A	[A]	–
1	pop A; visit; push C, B	[C, B]	A
2	pop B; visit; push D, C, A (A skipped)	[C, D, C]	A, B
3	pop C; visit; push B, A (both skipped)	[C, D]	A, B, C
4	pop D; visit; push E, B (B skipped)	[C, E]	A, B, C, D
5	pop E; visit; push F, D (D skipped)	[C, F]	A, B, C, D, E
6	pop F; visit; push E (skipped)	[C]	A, B, C, D, E, F
7	pop C; already visited	[]	A, B, C, D, E, F

DFS order: **A, B, C, D, E, F**. Notice DFS plunges deep (A → B → C, then A → B → D → E → F) before it finishes.

BFS Pseudocode

```

PROCEDURE BFS(graph, start)
    DECLARE queue : Queue
    DECLARE visited : Set

```

```

queue.enqueue(start)
ADD start TO visited
WHILE NOT queue.isEmpty() DO
    v ← queue.dequeue()
    OUTPUT v
    FOR EACH neighbour IN graph[v]
        IF neighbour NOT IN visited THEN
            ADD neighbour TO visited
            queue.enqueue(neighbour)
        ENDIF
    ENDFOR
ENDWHILE
ENDPROCEDURE

```

BFS Worked Trace — Starting from A

Step	Dequeue	Enqueue	Queue (front → rear)	Visited (output)
0	–	A	[A]	A
1	A	B, C	[B, C]	A, B, C
2	B	D	[C, D]	A, B, C, D
3	C	–	[D]	A, B, C, D
4	D	E	[E]	A, B, C, D, E
5	E	F	[F]	A, B, C, D, E, F
6	F	–	[]	A, B, C, D, E, F

BFS order: **A, B, C, D, E, F**. Although the visit order looks the same here, the *distances* differ: BFS discovers B and C at distance 1, D at distance 2, E at 3, F at 4 — a shortest-path tree.

DFS vs BFS Comparison

Aspect	DFS	BFS
--------	-----	-----

Frontier data structure	Stack (or recursion)	Queue
Space complexity	$O(h)$ where h is max recursion depth	$O(w)$ where w is max graph width
Time complexity	$O(V + E)$	$O(V + E)$
Finds shortest path (unweighted)?	No — may return a longer path first.	Yes — the first path found is shortest.
Good for...	Topological sort, cycle detection, maze solving, connected-component labelling.	Shortest unweighted path, level-order traversal, web-crawl within n hops, broadcast trees.
Risk	Can recurse very deep — stack overflow on huge graphs.	Queue can balloon on wide graphs.

Exam Tip: Examiners distinguish DFS from BFS by the data structure: **stack = DFS, queue = BFS**. Whenever you are asked to trace a traversal, always label which structure you are using and show its contents at each step.

Applications

- **DFS:** topological sort of a DAG (schedule tasks with prerequisites), detecting cycles, finding strongly-connected components (Tarjan / Kosaraju), solving mazes, parsing language grammars.
- **BFS:** shortest-path in unweighted graphs, level-order traversal of trees, finding the minimum number of moves in puzzles (15-puzzle, Rubik's), web crawlers limited by depth, network-broadcast simulation, peer-to-peer search.

4.3.2 Tree-Traversal Algorithms

For a binary tree the three *depth-first* orders differ in **when** the current node is visited relative to its left and right subtrees. All three have the same asymptotic cost — $O(n)$ time, $O(h)$ auxiliary stack space — but produce different orderings.

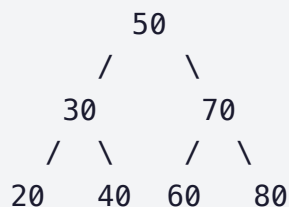
Pre-order: visit *Node*, then Left subtree, then Right subtree (NLR).

In-order: visit Left subtree, then *Node*, then Right subtree (LNR).

Post-order: visit Left subtree, then Right subtree, then *Node* (LRN).

Example BST

Insert 50, 30, 70, 20, 40, 60, 80:



Traversal Pseudocode

```

PROCEDURE preOrder(node)
  IF node ≠ NULL THEN
    OUTPUT node.key
    preOrder(node.left)
    preOrder(node.right)
  ENDIF
ENDPROCEDURE
  
```

```

PROCEDURE inOrder(node)
  IF node ≠ NULL THEN
    inOrder(node.left)
    OUTPUT node.key
    inOrder(node.right)
  ENDIF
ENDPROCEDURE
  
```

```

PROCEDURE postOrder(node)
  IF node ≠ NULL THEN
  
```

```

    postOrder(node.left)
    postOrder(node.right)
    OUTPUT node.key
ENDIF
ENDPROCEDURE

```

Traversal Results

Traversal	Order	Output on example tree	Typical application
Pre-order (NLR)	Node, Left, Right	50, 30, 20, 40, 70, 60, 80	Serialise a tree for reconstruction; prefix expression emission.
In-order (LNR)	Left, Node, Right	20, 30, 40, 50, 60, 70, 80	Emit BST contents in <i>sorted order</i> ; infix expression.
Post-order (LRN)	Left, Right, Node	20, 40, 30, 60, 80, 70, 50	Free / delete a tree safely (children before parents); postfix (RPN).

Connection to Expression Trees

An *expression tree* stores operators at internal nodes and operands at the leaves. Take the infix expression `(3 + 4) * (5 - 2)` :



Traversal	Output	Notation
Pre-order	* + 3 4 - 5 2	Prefix (Polish)
In-order	3 + 4 * 5 - 2	Infix (but loses parentheses)

Post-order	$3\ 4 + 5\ 2 - *$	Postfix (Reverse Polish)
------------	-------------------	--------------------------

Notice that in-order drops the parentheses, so $3 + 4 * 5 - 2$ is ambiguous without them. Pre-order and post-order are *unambiguous* without parentheses, which is why compilers and calculators convert to RPN before evaluation.

Exam Tip: A classic AQA question gives you a tree diagram and asks for one specific traversal. Circle each node in the correct order as you sweep through it; never try to read the answer off the tree diagram visually — always apply the recursive rule node-by-node.

4.3.3 Reverse Polish (Postfix) Notation

Reverse Polish notation (RPN), or *postfix*, writes each operator *after* its two operands: $3\ 4 +$ means $3 + 4$. RPN requires no parentheses and no operator-precedence rules — the structure is implicit in the order.

Advantages over infix: (1) **no parentheses required**, (2) **no precedence rules** required at evaluation time, (3) evaluation is a simple stack loop, (4) the ordering maps directly onto post-order traversal of an expression tree. RPN is used internally by compilers and by calculators such as the HP 48.

Evaluating RPN with a Stack

```

FUNCTION evaluateRPN(tokens) RETURNS Real
  DECLARE s : Stack
  FOR EACH t IN tokens
    IF isNumber(t) THEN
      s.push(t)
    ELSE
      // operator
      b ← s.pop()
      a ← s.pop()
      s.push(apply(t, a, b))
    ENDIF
  ENDFOR

```

```

RETURN s.pop()
ENDFUNCTION

```

Worked Evaluation — 5 1 2 + 4 * + 3 -

Token	Action	Stack after (bottom → top)
5	push 5	[5]
1	push 1	[5, 1]
2	push 2	[5, 1, 2]
+	pop 2, pop 1; push $1+2 = 3$	[5, 3]
4	push 4	[5, 3, 4]
*	pop 4, pop 3; push $3 \times 4 = 12$	[5, 12]
+	pop 12, pop 5; push $5+12 = 17$	[17]
3	push 3	[17, 3]
-	pop 3, pop 17; push $17-3 = 14$	[14]

Final answer: **14**. Infix equivalent: $(5 + (1 + 2) \times 4) - 3$.

Infix → Postfix: the Shunting-Yard Algorithm

Edsger Dijkstra's *shunting-yard* algorithm converts infix to postfix in one pass. It uses a stack of operators and an output queue.

```

FOR EACH token t IN infixExpression
  IF t is a number THEN
    output.enqueue(t)
  ELSE IF t is an operator o1 THEN
    WHILE opStack is not empty AND
      top(opStack) is an operator o2 AND
      (precedence(o2) > precedence(o1))

```

```

        OR (precedence(o2) = precedence(o1) AND o1 is left-associative) DO
            output.enqueue(opStack.pop())
        ENDWHILE
        opStack.push(o1)
    ELSE IF t is '(' THEN
        opStack.push(t)
    ELSE IF t is ')' THEN
        WHILE top(opStack) ≠ '(' DO
            output.enqueue(opStack.pop())
        ENDWHILE
        opStack.pop()          // discard the '('
    ENDIF
ENDFOR
WHILE opStack is not empty
    output.enqueue(opStack.pop())
ENDWHILE

```

Shunting-Yard Trace — $3 + 4 * 2 / (1 - 5)$

Token	Action	Operator stack	Output queue
3	output	[]	3
+	push +	[+]	3
4	output	[+]	3 4
*	prec(*) > prec(+): push *	[+, *]	3 4
2	output	[+, *]	3 4 2
/	prec(/) = prec(*), left-associative: pop *; push /	[+, /]	3 4 2 *
(	push (	[+, /, (]	3 4 2 *
1	output	[+, /, (]	3 4 2 * 1
-	top is (: push -	[+, /, (, -]	3 4 2 * 1
5	output	[+, /, (, -]	3 4 2 * 1 5

)	pop until (: pop −; discard (	[+, /]	3 4 2 * 1 5 −
end	pop remaining: /, then +	[]	3 4 2 * 1 5 − / +

Postfix output: **3 4 2 * 1 5 − / +**. Evaluating this with the stack loop above gives $3 + (4 * 2) / (1 - 5) = 3 + 8 / -4 = 3 + -2 = 1$.

Exam Tip: Two pitfalls AQA loves to test in RPN conversion: (1) the minus sign in $1 - 5$ gives a *negative* intermediate, so $8 / -4 = -2$ — watch your signs; (2) operators of *equal* precedence are left-associative in AQA, so the earlier one pops off first. Always state your assumed precedence and associativity.

4.3.4 Searching Algorithms

Linear Search

Scan the list from the first element to the last, comparing each to the target. Works on *any* list — sorted or unsorted, array or linked list.

```

FUNCTION linearSearch(list, target) RETURNS Integer
  FOR i ← 0 TO LENGTH(list) − 1
    IF list[i] = target THEN RETURN i ENDIF
  ENDFOR
  RETURN −1                // not found
ENDFUNCTION

```

Time complexity: $O(n)$ worst and average case; $O(1)$ best case (target at index 0). **Space:** $O(1)$. Use linear search whenever the list is small, unsorted, or changes constantly so sorting is not worthwhile.

Binary Search

Binary search finds a target in a *sorted* list by repeatedly comparing it with the middle element and discarding the half of the list that cannot contain the target. Each iteration

halves the search space, giving **$O(\log n)$** time.

```

FUNCTION binarySearch(list, target) RETURNS Integer
  low ← 0
  high ← LENGTH(list) - 1
  WHILE low ≤ high DO
    mid ← (low + high) DIV 2
    IF list[mid] = target THEN
      RETURN mid
    ELSE IF list[mid] < target THEN
      low ← mid + 1
    ELSE
      high ← mid - 1
    ENDIF
  ENDWHILE
  RETURN -1
ENDFUNCTION

```

Binary-Search Worked Trace

List: [3, 7, 11, 15, 19, 24, 28, 33, 41, 47, 55, 62, 70, 81, 92] (15 items, indices 0–14). Target: **33**.

Iteration	low	high	mid	list[mid]	Decision
1	0	14	7	33	Found — return 7

Lucky first hit. Now search for **19**:

Iteration	low	high	mid	list[mid]	Decision
1	0	14	7	33	$33 > 19 \rightarrow \text{high} \leftarrow 6$
2	0	6	3	15	$15 < 19 \rightarrow \text{low} \leftarrow 4$
3	4	6	5	24	$24 > 19 \rightarrow \text{high} \leftarrow 4$
4	4	4	4	19	Found — return 4

Now search for **50** (not present):

Iteration	low	high	mid	list[mid]	Decision
1	0	14	7	33	$33 < 50 \rightarrow \text{low} \leftarrow 8$
2	8	14	11	62	$62 > 50 \rightarrow \text{high} \leftarrow 10$
3	8	10	9	47	$47 < 50 \rightarrow \text{low} \leftarrow 10$
4	10	10	10	55	$55 > 50 \rightarrow \text{high} \leftarrow 9$
5	10	9	$\text{low} > \text{high} \rightarrow \text{return } -1$		

At most $\lceil \log_2 15 \rceil = 4$ comparisons are needed to decide presence. For a million items it is at most 20 comparisons — vastly better than a linear scan's million.

Exam Tip: Binary search has *two* preconditions: the list must be **sorted**, and it must support **random access** (i.e. it's an array, not a linked list). State both in any justification answer — AQA awards the mark for naming *both* requirements.

Binary-Tree Search

Searching a BST follows the BST property: at each node compare the target to the key, go left if smaller, right if larger. See 4.2.8 for full pseudocode. Complexity is $O(h)$: $O(\log n)$ for a balanced tree, $O(n)$ if the tree is pathologically skewed.

```

FUNCTION search(node, target) RETURNS Boolean
  IF node = NULL THEN RETURN False ENDIF
  IF node.key = target THEN RETURN True ENDIF
  IF target < node.key THEN
    RETURN search(node.left, target)
  ELSE
    RETURN search(node.right, target)
  ENDIF
ENDFUNCTION

```

Search-Algorithm Comparison

Algorithm	Data required	Best case	Average	Worst case	Space
Linear search	Any list	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary search	Sorted array	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iter / $O(\log n)$ rec
Binary-tree search	BST	$O(1)$	$O(\log n)$	$O(n)$ unbalanced	$O(h)$

4.3.5 Sorting Algorithms

Bubble Sort

Compare each adjacent pair from left to right; swap if the left is bigger than the right. Each full pass floats the largest remaining element to the end (like a bubble rising). Repeat until a complete pass performs no swaps.

```

PROCEDURE bubbleSort(a)
  n ← LENGTH(a)
  REPEAT
    swapped ← False
    FOR i ← 0 TO n - 2
      IF a[i] > a[i + 1] THEN
        SWAP a[i], a[i + 1]
        swapped ← True
      ENDIF
    ENDFOR
    n ← n - 1          // last element is now in place
  UNTIL swapped = False
ENDPROCEDURE

```

Bubble-Sort Worked Trace on [5, 3, 8, 1, 4]

Pass 1 (compare indices 0–3):

Compare	Array before	Swap?	Array after
5 vs 3	[5, 3, 8, 1, 4]	yes	[3, 5, 8, 1, 4]
5 vs 8	[3, 5, 8, 1, 4]	no	[3, 5, 8, 1, 4]
8 vs 1	[3, 5, 8, 1, 4]	yes	[3, 5, 1, 8, 4]
8 vs 4	[3, 5, 1, 8, 4]	yes	[3, 5, 1, 4, 8]

End of pass 1: [3, 5, 1, 4, 8] . Largest element 8 is in final position.

Pass 2 (compare indices 0–2):

Compare	Array before	Swap?	Array after
3 vs 5	[3, 5, 1, 4, 8]	no	[3, 5, 1, 4, 8]
5 vs 1	[3, 5, 1, 4, 8]	yes	[3, 1, 5, 4, 8]
5 vs 4	[3, 1, 5, 4, 8]	yes	[3, 1, 4, 5, 8]

End of pass 2: [3, 1, 4, 5, 8] .

Pass 3 (compare indices 0–1):

Compare	Array before	Swap?	Array after
3 vs 1	[3, 1, 4, 5, 8]	yes	[1, 3, 4, 5, 8]
3 vs 4	[1, 3, 4, 5, 8]	no	[1, 3, 4, 5, 8]

Pass 4 performs no swaps — done. Final array: [1, 3, 4, 5, 8].

Complexity: $O(n^2)$ comparisons and swaps in the worst and average case; $O(n)$ best case (already sorted, detected via the swapped flag). **Space:** $O(1)$ — in-place. **Stable:** yes.

Merge Sort

Merge sort is a *divide-and-conquer* algorithm: recursively divide the list in half until each sub-list has length 1, then *merge* adjacent sorted sub-lists back together. Runs in **$O(n \log n)$** time with **$O(n)$ auxiliary memory** in all cases, and is *stable*.

```

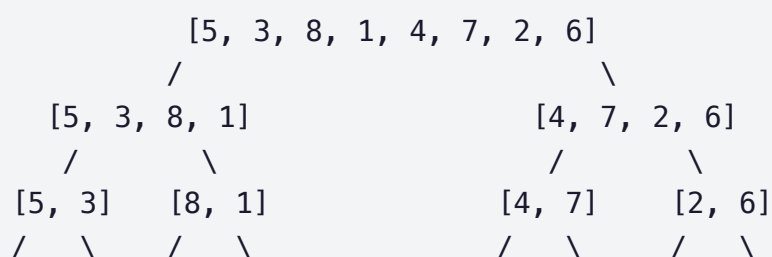
FUNCTION mergeSort(a) RETURNS List
  IF LENGTH(a) ≤ 1 THEN RETURN a ENDIF
  mid ← LENGTH(a) DIV 2
  left ← mergeSort(a[0 .. mid - 1])
  right ← mergeSort(a[mid .. LENGTH(a) - 1])
  RETURN merge(left, right)
ENDFUNCTION

FUNCTION merge(left, right) RETURNS List
  DECLARE result : List
  i ← 0 ; j ← 0
  WHILE i < LENGTH(left) AND j < LENGTH(right) DO
    IF left[i] ≤ right[j] THEN
      result.append(left[i]) ; i ← i + 1
    ELSE
      result.append(right[j]) ; j ← j + 1
    ENDIF
  ENDWHILE
  WHILE i < LENGTH(left) DO result.append(left[i]) ; i ← i + 1 ENDWHILE
  WHILE j < LENGTH(right) DO result.append(right[j]) ; j ← j + 1 ENDWHILE
  RETURN result
ENDFUNCTION

```

Merge-Sort Recursion Tree on [5, 3, 8, 1, 4, 7, 2, 6]

Divide phase (top-down):



[5] [3] [8] [1] [4] [7] [2] [6]

Merge phase (bottom-up):

[5] [3] → [3, 5] [4] [7] → [4, 7]
 [8] [1] → [1, 8] [2] [6] → [2, 6]
 [3, 5] [1, 8] → [1, 3, 5, 8]
 [4, 7] [2, 6] → [2, 4, 6, 7]
 [1, 3, 5, 8] [2, 4, 6, 7] → [1, 2, 3, 4, 5, 6, 7, 8]

The recursion tree has depth $\log_2 n$; each level does $O(n)$ work in total across all the merges, giving $O(n \log n)$ overall. The auxiliary list in `merge` uses $O(n)$ additional memory.

Why Merge Sort is Stable

A sort is *stable* if equal elements retain their original relative order. In the `merge` function the test `left[i] ≤ right[j]` uses $\leq$ (not $<$): when two keys are equal, the one from the *left* half is emitted first, preserving original order. A tiny change to $<$ would break stability.

Bubble Sort vs Merge Sort

Property	Bubble sort	Merge sort
Time (best)	$O(n)$	$O(n \log n)$
Time (average / worst)	$O(n^2)$	$O(n \log n)$
Extra memory	$O(1)$ — in-place	$O(n)$ — auxiliary array
Stable	Yes	Yes
Good for	Tiny inputs, teaching, almost-sorted data.	Large inputs; guaranteed $n \log n$ even on adversarial data; external sorting on disk.

Exam Tip: When AQA asks you to compare bubble sort and merge sort, hit *three* axes: time (n^2 vs $n \log n$), space ($O(1)$ vs $O(n)$), and practicality (bubble is fine for tiny lists; merge

scales). If the question mentions "very large dataset", always pick merge sort.

4.3.6 Optimisation Algorithms — Dijkstra's Shortest Path

Dijkstra's algorithm finds the shortest path from a single *source* vertex to every other vertex in a weighted graph with **non-negative** edge weights. It is a greedy algorithm: at each step it commits to the unvisited vertex with the currently smallest known distance.

Pseudocode (with a Priority Queue)

```
FUNCTION dijkstra(graph, source) RETURNS Dictionary
  FOR EACH v IN graph.vertices
    dist[v] ← ∞
    prev[v] ← NULL
  ENDFOR
  dist[source] ← 0
  DECLARE pq : PriorityQueue          // ordered by dist
  pq.insert(source, 0)

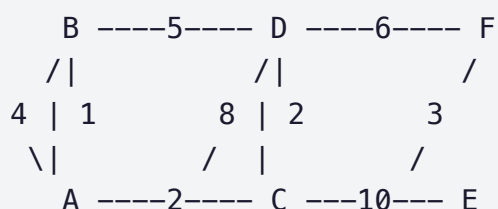
  WHILE NOT pq.isEmpty() DO
    u ← pq.extractMin()
    FOR EACH (v, w) IN graph.adj[u]   // edge u→v with weight w
      alt ← dist[u] + w
      IF alt < dist[v] THEN
        dist[v] ← alt
        prev[v] ← u
        pq.insertOrDecrease(v, alt)
      ENDIF
    ENDFOR
  ENDWHILE
  RETURN dist, prev
ENDFUNCTION
```

Example Weighted Graph

Vertices: A, B, C, D, E, F

Edges (undirected, weighted):

A-B : 4 A-C : 2
 B-C : 1 B-D : 5
 C-D : 8 C-E : 10
 D-E : 2 D-F : 6
 E-F : 3



Dijkstra Trace from Source A

Table columns: **V** = vertex just extracted from the priority queue; **PQ** = priority queue contents after processing (pairs of **vertex:distance**, sorted by distance); **dist** = current best distances; **prev** = predecessor on best-known path.

Step	V	Relaxed edges	PQ after	dist (A,B,C,D,E,F)
0	–	initialise	{A:0}	0, ∞, ∞, ∞, ∞, ∞
1	A (0)	A→B = 4, A→C = 2	{C:2, B:4}	0, 4, 2, ∞, ∞, ∞
2	C (2)	C→B = 2+1 = 3 (improves 4); C→D = 10; C→E = 12	{B:3, B:4, D:10, E:12}	0, 3, 2, 10, 12, ∞
3	B (3)	B→D = 3+5 = 8 (improves 10); B→A, B→C already finalised	{B:4, D:8, D:10, E:12}	0, 3, 2, 8, 12, ∞
4	B (4)	stale entry — B already finalised, skip	{D:8, D:10, E:12}	0, 3, 2, 8, 12, ∞

5	D (8)	$D \rightarrow E = 8 + 2 = 10$ (improves 12); $D \rightarrow F = 8 + 6 = 14$	{D:10, E:10, E:12, F:14}	0, 3, 2, 8, 10, 14
6	D (10)	stale, skip	{E:10, E:12, F:14}	0, 3, 2, 8, 10, 14
7	E (10)	$E \rightarrow F = 10 + 3 = 13$ (improves 14)	{E:12, F:13, F:14}	0, 3, 2, 8, 10, 13
8	E (12)	stale, skip	{F:13, F:14}	0, 3, 2, 8, 10, 13
9	F (13)	$F \rightarrow E$ already finalised, skip	{F:14}	0, 3, 2, 8, 10, 13
10	F (14)	stale, skip — PQ empty	{ }	0, 3, 2, 8, 10, 13

Final distances from A: **A = 0, B = 3, C = 2, D = 8, E = 10, F = 13**. Predecessors reconstruct the shortest-path tree: e.g. shortest $A \rightarrow F$ is $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$, which sums to $2 + 1 + 5 + 2 + 3 = 13$.

Time Complexity

Priority-queue implementation	Overall time	Notes
Unsorted array	$O(V^2)$	Good when the graph is dense ($E \approx V^2$).
Binary heap	$O((V + E) \log V)$	Standard choice for sparse graphs.
Fibonacci heap	$O(E + V \log V)$	Best theoretical; rarely used in practice due to constants.

Why Dijkstra Fails on Negative Weights

Dijkstra's correctness relies on the invariant that once a vertex is extracted from the priority queue, its shortest distance is final — because any future path through yet-unvisited vertices can only get *longer* (edges add non-negative weight). A negative edge could later shorten an already-finalised distance, violating the invariant. Counter-example:

```
A --1--> B
A --2--> C
C --(-5)--> B
```

Dijkstra from A extracts and finalises B first with $\text{dist}[B] = 1$, then extracts C ($\text{dist}[C] = 2$), and misses the cheaper path $A \rightarrow C \rightarrow B$ through the negative edge (which would give $\text{dist}[B] = 2 + (-5) = -3$).

When edges may be negative (but there are no negative cycles), use the **Bellman–Ford** algorithm. It relaxes every edge $V - 1$ times and runs in $O(VE)$; it also detects a negative cycle by checking if any edge can still be relaxed after $V - 1$ rounds.

A* — Dijkstra with a Heuristic

An **admissible heuristic** $h(n)$ is a function that estimates the cost from node n to the goal and **never overestimates** the true remaining cost. Admissibility guarantees that A* returns an optimal path.

A* search extends Dijkstra by ordering the priority queue by $f(n) = g(n) + h(n)$ rather than $g(n)$ alone, where $g(n)$ is the known cost from source to n and $h(n)$ is the admissible heuristic estimate of the remaining cost to the goal. With $h \equiv 0$, A* collapses back to Dijkstra. A typical admissible heuristic for geographic graphs is straight-line (Euclidean) distance — never an over-estimate because the true path along roads is at least as long.

Exam Tip: To trace Dijkstra in an exam: draw a table with one row per step, always extract the vertex with the smallest distance, and update distances with *strict* inequality ($\text{alt} < \text{dist}[v]$). Mark vertices as "settled" once extracted — examiners award marks for showing the settled set, not just the final distances.

Analysis and Design — Big-O Notation

Big-O notation describes an *upper bound* on the growth rate of a function. Formally $f(n) = O(g(n))$ means there exist constants $c > 0$ and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. In practice we use it to describe worst-case time or space complexity of an algorithm as a function of input size n .

Worst vs Average vs Best Case

- **Worst case** — the maximum number of basic operations over *all* inputs of size n . This is the guarantee we can offer. Most AQA questions want worst case.
- **Average case** — the expected number of operations over inputs drawn from some probability distribution (usually uniform). Harder to derive; sometimes matches the worst case (merge sort), sometimes much better (quicksort: $O(n \log n)$ average but $O(n^2)$ worst).
- **Best case** — the minimum over all inputs. Rarely informative on its own but useful to report (e.g. bubble sort is $O(n)$ best because an already-sorted array needs one clean pass).

Complexity Classes — Growth Ordering

From slowest-growing to fastest-growing (i.e. best-to-worst as $n \rightarrow \infty$):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Complexity Class Table with Examples

Class	Name	$n=10$	$n=10^3$	Example algorithm
$O(1)$	Constant	1	1	Hash-table look-up (average); array index; stack push.
$O(\log n)$	Logarithmic	3	10	Binary search; BST search (balanced); priority-queue operations on a heap.

$O(n)$	Linear	10	1 000	Linear search; sum of an array; BFS / DFS (counted per vertex).
$O(n \log n)$	Linearithmic	33	9 966	Merge sort; heap sort; Dijkstra with binary heap on sparse graphs.
$O(n^2)$	Quadratic	100	10^6	Bubble sort; insertion sort; naive matrix traversal; Dijkstra with array on dense graph.
$O(n^3)$	Cubic	1 000	10^9	Schoolbook matrix multiply; Floyd–Warshall all-pairs shortest paths.
$O(2^n)$	Exponential	1 024	$\approx 10^{301}$	Naive recursive Fibonacci; subset enumeration; exact TSP by brute force.
$O(n!)$	Factorial	3.6 M	intractable	Enumerating all permutations; brute-force travelling salesman.

Algorithm Selection Guide

Scenario	Recommended algorithm	Why
Find an item in an unsorted list of 100 elements.	Linear search	Sorting first would cost $O(n \log n)$ just to enable a single $O(\log n)$ look-up.
Find an item repeatedly in a sorted list of 10^6 elements.	Binary search	$O(\log n)$ each = ~ 20 comparisons vs $\sim 500\,000$ for linear.
Sort 1 000 000 records on a server.	Merge sort (or built-in Timsort)	Guaranteed $O(n \log n)$; stability preserved; parallelises well.
Teach sorting to beginners; sort 20 items.	Bubble sort	Trivial to explain; n is tiny so $O(n^2)$ is fine.

Shortest journey in a city road map with non-negative weights.	Dijkstra (A* with Euclidean heuristic)	Heuristic massively prunes explored area.
Shortest route when some edge weights may be negative (e.g. currency arbitrage).	Bellman–Ford	Dijkstra is incorrect on negative edges; Bellman–Ford also detects negative cycles.
Explore all reachable web pages within 3 hops of a seed URL.	BFS	Level-order exploration bounds depth naturally.
Schedule tasks with prerequisites (build system).	DFS + topological sort	Post-order of DFS on the DAG yields a valid ordering.

Space Complexity

Space complexity counts *auxiliary* memory beyond the input. Key numbers to remember:

- **Bubble sort, linear search, iterative binary search:** $O(1)$ — in-place / no extra structure.
- **Recursive binary search, BST search, recursive tree traversal:** $O(h)$ stack space where h is depth.
- **Merge sort:** $O(n)$ auxiliary array (cannot do better with this algorithm).
- **BFS:** $O(V)$ for the queue and visited set.
- **DFS (recursive):** $O(V)$ worst case for the recursion stack.
- **Dijkstra:** $O(V)$ for dist, prev, and priority queue.

Exam Tip: When AQA asks for "time complexity of X", quote *Big-O worst case* unless the question asks for best or average. Always justify briefly: " $O(\log n)$ because each iteration halves the remaining search space" earns more marks than the symbol alone.

Algorithm Summary Table — All the Named Algorithms

Algorithm	Time (best)	Time (avg)	Time (worst)	Space	Spec section
-----------	-------------	------------	--------------	-------	--------------

DFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$	4.3.1
BFS	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$	4.3.1
Pre/In/Post-order	$O(n)$	$O(n)$	$O(n)$	$O(h)$	4.3.2
RPN evaluation	$O(n)$	$O(n)$	$O(n)$	$O(n)$	4.3.3
Linear search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	4.3.4
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	4.3.4
Binary-tree search	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$	4.3.4
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	4.3.5
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	4.3.5
Dijkstra (bin. heap)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$	4.3.6
Bellman–Ford	$O(VE)$	$O(VE)$	$O(VE)$	$O(V)$	4.3.6 (extension)
A*	depends on heuristic	$O(b^d)$ worst	$O(b^d)$	$O(b^d)$	4.3.6 (extension)

Summary Checklist

- Trace DFS (stack) and BFS (queue) on the same small graph, showing the frontier contents at every step.
- Quote $O(V + E)$ time for both traversals and state the defining data-structure difference (stack vs queue).

- Give pre-order, in-order and post-order outputs for a given binary tree, connecting them to prefix / infix / postfix expression notation.
- Convert an infix expression to Reverse Polish using the shunting-yard algorithm; evaluate RPN with a stack.
- Implement and trace linear search ($O(n)$) and binary search ($O(\log n)$); state binary search's preconditions (sorted, random-access).
- Describe binary-tree search and its dependence on tree height.
- Trace bubble sort pass-by-pass and merge sort as a recursion tree; state complexity classes and stability.
- Trace Dijkstra's algorithm on a small weighted graph showing the priority queue and distance table at every step.
- Explain why Dijkstra fails on negative edge weights and name Bellman–Ford as the replacement.
- Define an admissible heuristic and describe how A* extends Dijkstra.
- State the formal definition of Big-O and list complexity classes from $O(1)$ to $O(n!)$.
- Pick the right algorithm for a given scenario and justify in terms of both time and space complexity.